

From Policy Semantics to Safe Tool Execution: Action–Evidence Geometry and Execution Certificates

Anonymous submission

Abstract

A well-formed tool call can still be unsupported by the trusted task, evidence, authorization, or current state. We present DAFR, an execution-layer compiler that represents each proposed call as an action–evidence point and evaluates it against state-dependent halfspaces and joint-risk regions. A deterministic typed field interface preserves the link from schema arguments to shared effect roles; the encoder records grounding, provenance, authorization, and risk coordinates; and the verifier emits a certificate with signed margins and violated relations. On the evaluated static attacks, DAFR obtains 0/585 observed attack successes on AgentDojo and 0/204 on Agent Security Benchmark, with 87.5% clean utility and 81.4% attack utility on AgentDojo and 86.8% utility on ASB. A matched 512-case study finds 25 false allows for independent axis thresholds, while an equivalent arithmetic predicate agrees with the geometric boundary on all cases. DAFR connects policy-defined joint risk to a common decision certificate, making the limiting execution relations explicit and available for action revision.

Introduction

Tool-using agents can read private information, modify shared resources, contact external parties, and trigger actions that are difficult to reverse. These calls have more direct consequences than generated text: an executed call may immediately disclose information, change system state, or start an external operation (Yao et al. 2023; Schick et al. 2023; Ruan et al. 2024; Ye et al. 2024; Lee et al. 2026). Safe execution requires more than a valid schema. The function, effect-bearing arguments, expected effect, and state dependencies must be supported by the current task, available evidence, authorization, and policy.

Indirect prompt injection makes this decision difficult (Greshake et al. 2023; Liu et al. 2023; Zhan et al. 2024). An agent may need untrusted content to complete a valid task, yet that content can contain attacker-inserted instructions. The model can translate them into a well-formed, apparently relevant call that discloses sensitive information, modifies the wrong object, or creates an unauthorized external effect

(Debenedetti et al. 2024; Liao et al. 2025; Wang et al. 2025; Li et al. 2026; Ma et al. 2026). Tool descriptions and environment metadata provide additional channels for malicious control (Wang et al. 2026). The execution-time question is whether trusted evidence supports the call’s effect in the current state.

Consider an agent asked to update a customer record whose identifier is initially unknown. The agent retrieves the record, obtains its identifier from the verified result, and uses that identifier for the requested update. The retrieved content may also contain an attacker-issued command to send the record to a newly specified address. Both actions are schema-valid, but their support differs. The update follows the user request and uses an identifier established by the verified retrieval. The disclosure relies on a destination introduced through untrusted content and produces an unauthorized external effect. The execution layer must distinguish these actions by the evidence supporting their concrete effects.

Existing defenses act at different stages of the agent pipeline. Context-level methods separate trusted instructions from untrusted data, restructure prompts, repeat the user request, or filter suspicious observations before action generation (Hines et al. 2024; Wallace et al. 2024; Jia et al. 2025; Chen et al. 2025). Runtime methods apply symbolic policies, privilege checks, execution plans, or information-flow rules to the generated action (Wang, Poskitt, and Sun 2025; Shi et al. 2025; Li et al. 2025a; Debenedetti et al. 2025). Tool-action feasibility, however, often depends on relations among the action, its evidence, and the current state. A destination can be valid when specified by the user and invalid when copied from a web page. A write can become feasible after a verified read establishes its target. Combining sensitive data with an external destination can require stronger grounding and authorization than either factor alone.

We introduce DAFR (Dynamic Action Feasible Regions), an execution-layer compiler for deciding whether a concrete tool action is feasible at the current step. A deterministic typed field interface binds registered schema fields to shared effect roles and validates the policy envelope. An action–evidence encoder then grounds the call, runtime state, and source-tagged evidence in these roles. State-dependent halfspaces and joint-risk regions evaluate the resulting point and emit an execution certificate: a call inside the region

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

proceeds, while a call outside it returns violated relations for clarification, repair, or blocking. This interface connects language-model proposals to explicit execution requirements and preserves the evidence needed to inspect each decision.

The certificate changes with execution. A verified read can establish an identifier required by a later write, a trusted confirmation can authorize a scoped effect, and a new observation can invalidate an earlier state assumption. DAFR reconstructs the action–evidence point and active constraints before every external effect, retaining original field names and evidence links in the trace. The contribution is the composition of typed grounding, action–evidence encoding, joint-risk verification, and a certificate interface that exposes the limiting relation used for the next decision.

We evaluate DAFR on AgentDojo with three planner models and on the Agent Security Benchmark (ASB) with DeepSeek V4 Flash. DAFR obtains 0/585 and 0/204 observed attack successes on the two benchmarks, retaining 87.5% clean utility and 81.4% attack utility on AgentDojo and 86.8% utility and 73.5% task success on ASB. A system ablation shows that removing the complete dynamic defense path increases observed ASR to 16.7%; a separate matched study isolates the narrower representation claim: independent axis checks miss coupled budgets, while an equivalent predicate agrees with the joint boundary and the geometric certificate exposes the limiting relations used for diagnosis and repair.

DAFR: Dynamic Action Feasible Regions

DAFR constructs an interpretable action–evidence point for the proposed call and instantiates the geometric constraints active in the current state. Region membership determines feasibility, and signed margins identify the relations that limit the decision.

Problem Setup

At step t , a tool-using agent receives a trusted user task u , operates under an execution policy p , observes the environment, and produces a concrete tool call

$$a_t = (f_t, \rho_t), \quad (1)$$

where f_t is a registered tool and ρ_t maps the fields of the tool schema to argument values. The call is first checked against the schema. Passing this check establishes that the call is well formed and leaves its execution feasibility to the evidence and policy checks below.

Let E_t denote the execution state before a_t is evaluated. It contains the trusted task, execution policy, available tool schemas, established facts, completed prerequisites, authorizations, confirmations, and observations with source labels. Facts derived from external observations retain their source labels, while intent, confirmation, and authorization are recorded through trusted channels.

A call is feasible when the current evidence and policy support its function, the arguments that determine its external effect, the expected effect itself, and the required state conditions. We write

$$\text{Feas}(a_t \mid E_t, u, p) = 1 \quad (2)$$

when these conditions are satisfied. A feasible call is sent to the trusted runtime. An infeasible call is blocked or returned with feedback that identifies the function, argument, evidence source, authorization condition, or state assumption requiring revision. After an admitted call returns observation o_{t+1} , the state is updated as

$$E_{t+1} = T(E_t, a_t, o_{t+1}), \quad (3)$$

where T may add evidence, satisfy a prerequisite, record authorization, or invalidate an earlier assumption.

Typed Action–Evidence Compiler

The compiler separates typed field validation, action–evidence encoding, and execution verification. Let B_f bind the fields of registered tool f to a shared set of effect roles, let E encode a concrete call and its current evidence, and let G verify the resulting point. For a policy p and schema S_f , deterministic validation constructs

$$\hat{B}_f = \text{Validate}(p, S_f), \quad (4)$$

where the interface records supported field roles, required prerequisites, authorization scope, and risk families. Unknown fields, unsupported effect types, inconsistent requirements, and out-of-range budgets are rejected before execution. Given a concrete call a_t and state E_t , the encoder computes

$$z_t = E(a_t, E_t, \hat{B}_f, p), \quad (5)$$

retaining original field names and source links in the trace. The verifier returns

$$(d_t, \kappa_t) = G(z_t, E_t, p), \quad (6)$$

where d_t is the execution decision and κ_t contains the active constraints, signed margins, provenance links, and revision targets. The three stages are therefore an auditable interface from a concrete language-model proposal to a deterministic execution decision.

Action–Evidence Point Construction

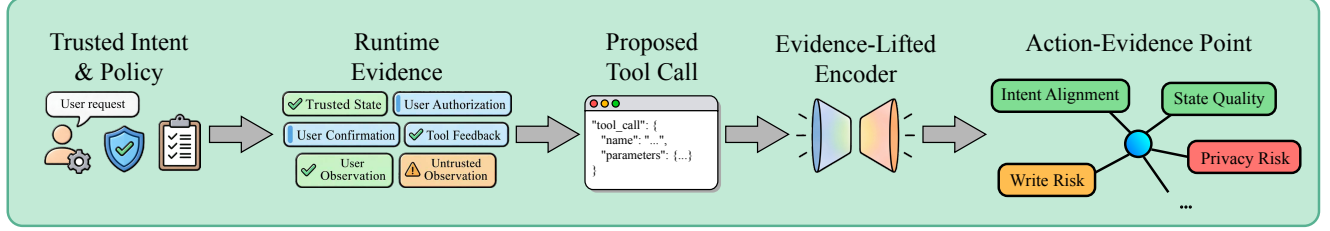
DAFR represents each proposed tool call and its current evidence as a point in an interpretable action–evidence space. The coordinates cover task support and progress, argument grounding and provenance, authorization and scope, and expected effects and risks.

DAFR uses the validated typed interface and action–evidence encoder to construct the action–evidence point:

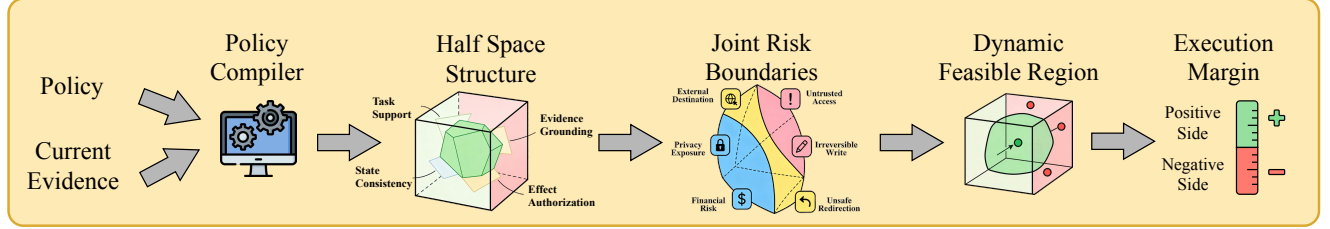
$$z_t = L(a_t, E_t, u, p) = (q_t, g_t, c_t, r_t) \in [0, 1]^d, \quad (7)$$

where L deterministically evaluates explicit execution relations. The runtime uses seven shared execution roles (object, destination, data, amount, time, effect, and scope) and a 45-dimensional action–evidence vector. Its compiler exposes 13 hard constraint templates and one soft confidence facet; coefficients and risk budgets are fixed globally for the evaluated configuration, while active role bindings and templates depend on the registered schema and current effect. A new tool therefore requires schema validation and an auditable field binding, not a new set of fitted geometric weights. The task

Step 1: Build the Action-and-Evidence Point



Step 2: Compile the Dynamic Feasible Region



Step 3: Decide, Execute, and Update

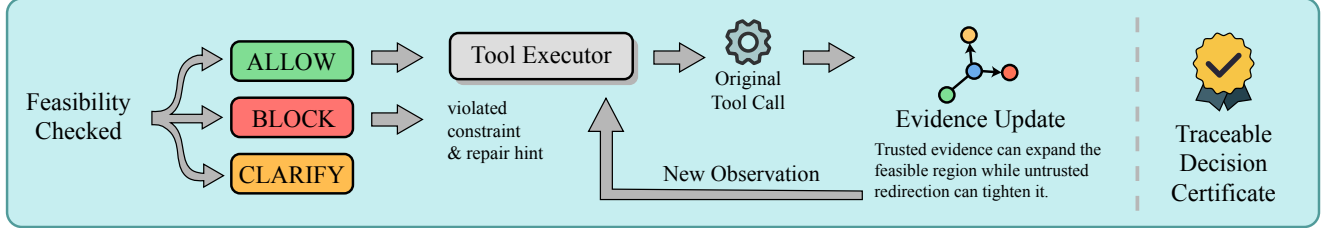


Figure 1: DAFR turns a proposed tool call, source-tagged evidence, and current state into an action–evidence point. Typed field validation preserves effect roles; the encoder records grounding, authorization, provenance, and risk; and the verifier evaluates active halfspaces and joint-risk regions. Region membership determines execution, while violated margins identify the relation used for clarification, repair, or blocking.

block q_t records support for the selected function and its expected effect. The grounding block g_t connects effect-bearing arguments to source-tagged evidence and established state. The authorization block c_t records confirmation, authority, and scope. The risk block r_t describes disclosure, transfer, deletion, redirection, and state change.

For an effectful call, the typed interface preserves the original field, value, grounding score, and source label. A destination introduced only by an untrusted observation therefore receives different provenance and grounding values from a destination specified by the trusted task, even when both calls satisfy the tool schema. This shared representation is the input to every comparison in the matched geometry study.

The evidence used by these relations is maintained in the execution state. After an admitted call returns o_{t+1} , an evidence projection operator converts it into source-tagged factual and risk records:

$$P(o_{t+1}) = (S_{t+1}^+, R_{t+1}^+), \quad (8)$$

where S_{t+1}^+ contains entities, argument values, completed prerequisites, and observed state changes, and R_{t+1}^+ contains untrusted instructions, task conflicts, unsupported effect requests, and other risk signals. Each record retains its source,

trust status, and execution role when incorporated into E_{t+1} . Factual records update grounding and state relations, risk records update source, conflict, and effect-risk relations, and trusted records establish intent, confirmation, and authorization.

Each coordinate has a defined execution meaning and takes a value in $[0, 1]$ under the current call and execution state. Coordinates for prerequisites and authorization record whether the corresponding conditions have been established, while grounding and scope coordinates record the support available for a specific argument or effect from source-tagged evidence. For example, a verified lookup can establish the identifier required by a later update, whereas a destination introduced by untrusted content receives no authorization support for an external disclosure.

For an effectful call, let $V(a_t)$ denote the argument values that directly determine its external effect, such as a destination, object identifier, amount, date, or sensitive data. DAFR evaluates each value against the trusted task, source-tagged evidence, and current state. The critical grounding score is

$$g_{\text{crit}}(a_t, E_t) = \min_{v \in V(a_t)} g(v, E_t), \quad (9)$$

where $g(v, E_t) \in [0, 1]$ measures the available support for

value v . The minimum gives a conjunctive grounding summary across the arguments that determine the external effect: every critical value must satisfy its grounding requirement. Schema-optional fields follow the same effect-aware check. A field is eligible for omission during revision only when its removal leaves the destination, object, data, scope, authorization requirements, and expected external effect unchanged. Otherwise, an unsupported value keeps the call infeasible.

As evidence and execution state evolve, DAFR updates the affected coordinates, so the same proposed call may occupy a different position at a later execution step.

Dynamic Geometric Constraints

DAFR converts the execution requirements at step t into geometric regions in the action–evidence space. Each policy is compiled from the validated typed interface into constraint templates and coefficients before evaluation. The runtime parameters are frozen after compilation and are not tuned on the evaluated trajectory. For a proposed call, its expected effect, available evidence, authorization scope, policy, and current state select the active templates. Let $\mathcal{H}_t = \mathcal{H}(a_t, E_t, u, p)$ denote requirements expressed through individual execution relations. Each $j \in \mathcal{H}_t$ defines a halfspace

$$\mathcal{P}_j = \{z \in [0, 1]^d \mid m_j^{\text{half}}(z) = b_j - w_j^\top z \geq 0\}. \quad (10)$$

For each active requirement, the policy compiler retrieves w_j and b_j from the corresponding fixed template. The vector w_j selects and weights the relevant execution relations, and b_j sets the feasibility boundary. The coefficient signs follow the semantic direction of each relation: stronger trusted support raises the margin, and greater execution risk lowers it. These halfspaces represent task support, completed prerequisites, state consistency, critical-argument grounding, destination validity, confirmation, and authorization.

Some conditions depend on several relations at once. Let $\mathcal{K}_t = \mathcal{K}(a_t, E_t, u, p)$ denote the joint-risk conditions active at step t . Each $k \in \mathcal{K}_t$ defines a joint margin and its feasible region:

$$m_k^{\text{joint}}(z) = \beta_k + v_k^\top C_k z - \|W_k R_k z\|_2, \quad \mathcal{C}_k = \{z \in [0, 1]^d \mid m_k^{\text{joint}}(z) \geq 0\}. \quad (11)$$

The template for risk family k specifies the selectors C_k and R_k , the support weights v_k , the risk weights W_k , and the base risk budget β_k . The selectors extract the support and risk coordinates used by the condition. The support and risk weights are nonnegative, preserving the semantic direction of the joint margin. The curved boundary makes the acceptable level of each risk depend on the other risks present and the trusted support available for the action.

The feasible region is the intersection of all applicable regions:

$$\mathcal{F}_t = \left(\bigcap_{j \in \mathcal{H}_t} \mathcal{P}_j \right) \cap \left(\bigcap_{k \in \mathcal{K}_t} \mathcal{C}_k \right). \quad (12)$$

Only points that satisfy every relevant requirement belong to $\mathcal{F}_t$. DAFR instantiates this geometry at every step through

the current z_t , $\mathcal{H}_t$, and $\mathcal{K}_t$. New evidence can move the point, introduce relevant boundaries, or satisfy conditions that previously prevented execution.

The running example shows how this geometry changes over a trajectory. Before the lookup returns, the update lacks the record identifier required by its grounding and prerequisite relations. The verified lookup establishes that value and allows the update point to enter the feasible region. The same observation provides no authority for an unrelated disclosure. That action combines an untrusted destination, sensitive data, and an external effect without matching authorization, so it remains outside $\mathcal{F}_t$. Evidence can therefore support one concrete effect while leaving another infeasible because the calls depend on different roles, sources, and authorization conditions.

Geometric Properties

Convexity and semantic direction. For a fixed execution context, every $\mathcal{P}_j$ is a halfspace. Each joint margin in Eq. (11) is concave because it is an affine function minus a norm, so its superlevel set $\mathcal{C}_k$ is convex. Their intersection $\mathcal{F}_t$ is therefore convex and defines one coherent feasible region for the current decision.

The parameter signs preserve a common semantic direction across the geometry. Trusted grounding, confirmation, and authorization increase the relevant margins, while externality, sensitivity, irreversibility, and untrusted control decrease them. Support and risk therefore move an action toward opposite sides of the active boundaries.

Joint risks. The curved regions capture how the acceptable level of one risk changes with the other risks present and with the trusted support available for the action. When multiple risk relations share one authorization budget, the norm term in Eq. (11) permits different allocations while limiting their combined magnitude. An increase in one risk reduces the amount of the others that can be accepted under the same evidence and authorization.

What the representation changes. Independent coordinate guards implement an axis-aligned region, $\mathcal{A} = \{z : z_i \leq \tau_i \ \forall i\}$. A joint-risk constraint instead admits z only when the coupled risk budget is satisfied. Thus a point can lie in $\mathcal{A}$ while violating a shared budget, which is the failure mode tested by the matched suite. An unrestricted arithmetic predicate can implement the same joint boundary; our equivalent-predicate arm is required to agree with geometry. The claimed benefit of geometry is therefore the shared certificate interface—active facets, signed margins, provenance links, and repair targets—rather than greater decision expressiveness.

Why a shared boundary helps. Consider the illustrative budget $r_1^2 + r_2^2 \leq 1$. Independent guards with thresholds 0.8 accept (0.75, 0.75) even though its joint norm exceeds one. Tightening both thresholds to $1/\sqrt{2}$ removes this failure but rejects (0.8, 0.2), which satisfies the joint budget. A coupled boundary represents this allocation trade-off directly. The example explains a policy representation choice; it does not establish that an ℓ_2 budget is the correct safety policy for

every tool. A predicate implementing the same inequality makes the same decision. DAFR additionally associates the boundary with evidence links and revision targets through its certificate interface.

Source-aware state changes. New records update the relations they establish. A verified read can increase grounding and prerequisite support for returned values; a trusted confirmation can increase scoped authorization for a specified effect, object, or destination; and an untrusted observation can contribute factual values while activating source, conflict, or redirection risks. These source-aware updates allow one observation to support a requested write while leaving an unrelated disclosure outside the feasible region.

Feasibility Decision and Targeted Feedback

Given z_t , DAFR evaluates every margin that defines $\mathcal{F}_t$. The overall feasibility margin is

$$M_t(z) = \min \left\{ \min_{j \in \mathcal{H}_t} m_j^{\text{half}}(z), \min_{k \in \mathcal{K}_t} m_k^{\text{joint}}(z) \right\}. \quad (13)$$

Because membership in $\mathcal{F}_t$ requires every active margin to be nonnegative,

$$z_t \in \mathcal{F}_t \iff M_t(z_t) \geq 0. \quad (14)$$

The sign of $M_t(z_t)$ is equivalent to region membership, and its magnitude records the raw signed slack under the most limiting active condition. DAFR executes the call when $M_t(z_t) \geq 0$. When an admissible revision path exists, a failed condition produces targeted feedback for the planner. All other infeasible calls are blocked.

A call may violate several boundaries. DAFR records

$$\mathcal{V}_t = \{c \in \mathcal{H}_t \cup \mathcal{K}_t \mid m_c(z_t) < 0\}, \quad (15)$$

where m_c denotes the halfspace or joint margin associated with condition c . For feedback ordering, DAFR places violated margins on comparable scales through $\bar{m}_c(z_t) = m_c(z_t)/\sigma_c$, with $\sigma_c > 0$. Raw margins continue to determine feasibility. The most limiting conditions are selected from $\mathcal{V}_t$, and $\phi_t = \Gamma(\mathcal{V}_t, z_t, E_t)$ maps them to concise revision targets. Feedback can identify an unsupported tool choice or argument, an unsuitable source, missing authorization, an incomplete prerequisite, an outdated state assumption, or the factors contributing to a joint-risk violation.

Each decision records the concrete call, its action–evidence point, the active boundaries, the margin vector, and the supporting evidence links. This record provides the same geometric basis for the execution decision and the revision target.

Runtime Update and Cost

Each invocation of DAFR evaluates one concrete proposed call. A feasible call is executed by the trusted runtime, and the returned observation is projected into source-tagged records and incorporated into the next execution state through Eqs. (8) and (3). An infeasible call produces no external effect. When targeted feedback is returned, the planner may revise the call, obtain additional evidence, request authorization, or end the task. Any subsequent proposal is evaluated as a new call under the state available at that step.

Algorithm 1 DAFR Execution-Step Decision

Input: proposed call $a_t = (f_t, \rho_t)$, state E_t , task u , policy p .

Output: execution decision and feedback.

- 1: Check a_t against the registered schema of f_t .
 - 2: Validate registered fields and bind them to shared effect roles.
 - 3: Build $z_t = L(a_t, E_t, u, p)$ from the current relations.
 - 4: Instantiate $\mathcal{H}_t$ and $\mathcal{K}_t$ from the active policy templates.
 - 5: Evaluate all active margins and compute $M_t(z_t)$.
 - 6: **if** $M_t(z_t) \geq 0$ **then**
 - 7: Execute a_t , obtain o_{t+1} , and update E_{t+1} .
 - 8: **else**
 - 9: Form $\mathcal{V}_t$ and derive feedback ϕ_t .
 - 10: **if** an admissible revision path exists **then**
 - 11: Return CLARIFY with ϕ_t to the planner.
 - 12: **else**
 - 13: Return BLOCK.
 - 14: **end if**
 - 15: **end if**
-

Runtime cost and local updates. Let $C_t = |\mathcal{H}_t| + |\mathcal{K}_t|$ denote the number of geometric regions evaluated at step t , and let s_c denote the number of coordinates used by region c . Computing the complete margin vector requires

$$O\left(\sum_{c=1}^{C_t} s_c\right) \leq O(C_t d) \quad (16)$$

operations. Each region uses only a subset of the execution relations, so margin evaluation scales with the active regions and the coordinates they select.

Observations from admitted tool executions or trusted user interactions update only the relations linked to the affected values, sources, authorization conditions, and state facts. Before evaluating the next proposed external effect, DAFR constructs its action–evidence point and reevaluates every applicable margin. Local relation updates and complete margin reevaluation limit computation to the active regions and their selected coordinates while preserving a full feasibility check for each proposal.

Evaluation

Experimental Setup

AgentDojo v1.2.2 is the primary benchmark and contains the Banking, Slack, Travel, and Workspace suites (Debenedetti et al. 2024). We evaluate DeepSeek V4 Flash, GPT-5.4-mini, and Claude Haiku 4.5 on a fixed split of 71 clean and 585 attacked episodes: 16/144 in Banking, 21/105 in Slack, 20/140 in Travel, and 14/196 in Workspace. Clean and attack scores are unweighted macro-averages over suites. Figure 2 shows representative baselines and the supplement reports the complete table and case counts. Component ablations evaluate evidence projection, action–evidence encoding, the dynamic defense path, and decision feedback. A blocked or malformed call remains in the benchmark denominator. For 0/585 and 0/204 observed attacks, Wilson 95% upper bounds are 0.65% and 1.85%, respectively.

We further evaluate DAFR on 204 ASB cases using DeepSeek V4 Flash (Zhang et al. 2025). AgentDojo reports clean utility, attack utility, and attack success rate (ASR), while ASB reports utility, task success, and ASR.

DAFR is placed at the execution boundary, after the planner proposes a concrete tool call and before the trusted runtime performs its external effect. The reported comparisons target matched case sets and planner models. The archived GPT baseline aggregates combine earlier three-suite document summaries with Workspace reruns; they are not uniformly fresh four-suite reruns. We disclose this provenance in the supplement and interpret differences descriptively. Under this setup, DAFR contributes the feasibility decision and relation-specific revision feedback for the proposed call, while the benchmark task and planning environment remain unchanged.

Main Results on AgentDojo

Figure 2 reports the AgentDojo comparison across three planner models. With DeepSeek V4 Flash, DAFR achieves 87.5% clean utility and 81.4% attack utility with 0% observed ASR under the evaluated static attacks. DAFR also records 0% observed ASR under the evaluated static attacks with GPT-5.4-mini and Claude Haiku 4.5, while retaining 59.5% and 68.4% attack utility, respectively.

PIGuard is the only other representative method that reaches 0% observed ASR under the evaluated static attacks with all three planners. At this security level, DAFR improves attack utility over PIGuard by 38.9, 25.1, and 30.2 percentage points on DeepSeek, GPT, and Claude, respectively. These fixed-set comparisons show a favorable security–utility trade-off against PIGuard. They do not establish uniform dominance: with GPT, DAFR has lower attack utility than Prompt (65.4%) and Sandwich (65.4%), which have nonzero observed ASR; with Claude, Sandwich also has zero observed ASR and 67.1% attack utility, compared with 68.4% for DAFR.

Results on Agent Security Benchmark

Table 1 reports results on 204 ASB cases with DeepSeek V4 Flash. DAFR achieves 86.8% utility and 73.5% task success with 0% observed ASR under the evaluated static attacks.

Method	Utility	Task Succ.	ASR
No Defense	88.7	77.9	90.2
Delimiter	87.3	77.0	86.3
Instruction Prevention	88.0	77.5	39.7
Observation Sandwich	88.0	77.0	27.9
TS-Flow	85.5	73.5	19.1
Progent	86.0	73.0	3.4
DRIFT	85.8	73.0	23.0
DAFR	86.8	73.5	0.0

Table 1: Results on 204 ASB cases with DeepSeek V4 Flash. Values are percentages; DAFR has 0/204 observed attack successes.

Compared with no defense, DAFR reduces ASR from 90.2% to 0%, with a 1.9-point difference in utility. Among

the evaluated methods, DAFR is the only one that reaches 0% observed ASR under the evaluated static attacks. Relative to Progent, the strongest competing method by ASR, DAFR gains 0.8 points in utility and 0.5 points in task success and reduces ASR from 3.4% to 0%. The ASB results extend DAFR’s security–utility balance beyond AgentDojo to a distinct tool-use benchmark.

Ablation and Geometric Analysis

Table 2 reports the full AgentDojo system ablation with DeepSeek V4 Flash: each configuration uses the same 71 clean and 585 attacked cases across four suites, with suite-macro-averaged scores. These rows are system-level ablations because projection, encoding, geometry, and feedback interact. Removing evidence projection lowers clean utility from 87.5% to 79.2% while retaining 81.8% attack utility and 0% observed ASR under the evaluated static attacks. Removing action–evidence encoding raises clean and attack utility to 91.5% and 85.3% but introduces a 0.7% observed ASR.

Removing the complete dynamic defense path produces the largest security change in the original module ablation: observed ASR rises from 0% to 16.7%, with attack utility at 81.3%. This system ablation also changes observation handling and feedback, so it does not isolate the syntax of geometry. We therefore use a matched representation ablation: The membership controls share risk vectors and the repair controls share typed actions, evidence, and candidate transformations. The general predicate and geometric implementations are expected to agree on membership; the scientific comparison concerns axis-threshold expressiveness, margin-based diagnosis, and executable repair, not a claim that geometry can express functions unavailable to unrestricted programs. In a frozen 512-case joint-risk suite, independent axis thresholds allowed all cases and falsely allowed 25 actions whose joint risk exceeded the shared budget; the joint cone and equivalent predicate both allowed 487/512 and disagreed on 0/512. In a matched 256-case action suite, Geometry and Predicate+oracle each recovered 192 executable actions, and all 64 non-repairable actions remained blocked. A decision-only interface has no repair operator, so its zero recovery is a design property rather than an independently measured performance gap. This is evidence about coupled-budget encoding and an auditable repair interface, not arbitrary-program superiority. Removing decision feedback lowers clean and attack utility to 79.4% and 80.0% and introduces a 0.8% observed ASR.

Configuration	Clean	Attack	ASR
DAFR	87.5	81.4	0.0
w/o Evidence Projection	79.2	81.8	0.0
w/o Action–Evidence Encoding	91.5	85.3	0.7
w/o Dynamic Defense Path	90.2	81.3	16.7
w/o Decision & Feedback	79.4	80.0	0.8

Table 2: Component ablation on AgentDojo. Values are percentages.

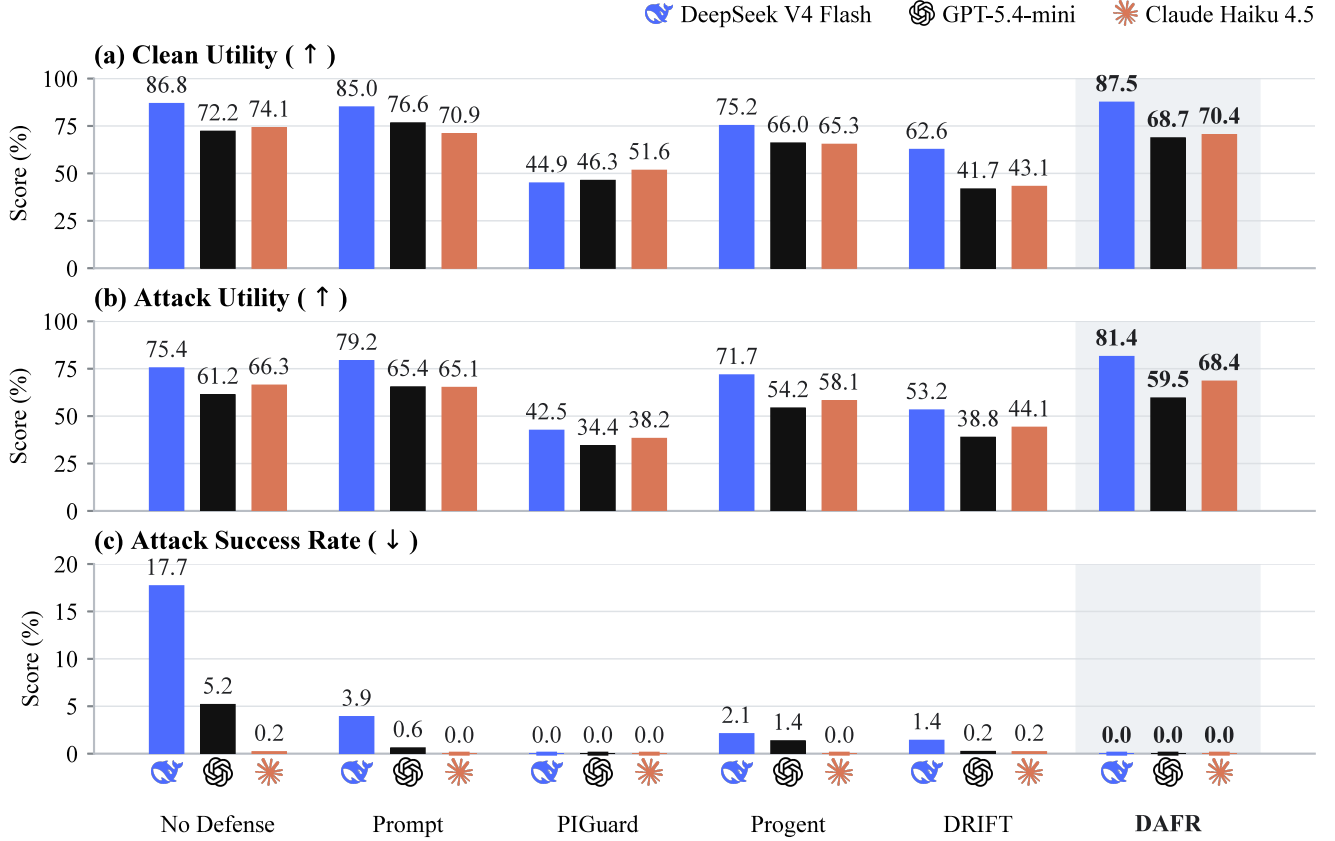


Figure 2: AgentDojo clean utility, attack utility, and attack success rate across six representative methods and three planner models. Values are percentages; arrows indicate the preferred direction. The supplement reports all nine evaluated methods.

Matched geometry protocol. We separate membership and repair controls. These matched controls are locally generated mechanism tests, rather than additional AgentDojo episodes: the 512 membership vectors are generated with fixed seed 20260930 in four dimensions, with every coordinate below the independent threshold 0.8 and analytic labels from the shared budget $\|r\|_2 \leq 1$ (25 vectors violate this joint budget); the 256 repair cases are generated from a fixed local-write effect model. The membership study supplies identical risk vectors to independent axis thresholds, the joint-risk region, and an arithmetic predicate implementing the same joint boundary. The repair study supplies identical actions, evidence, and candidate transformations to the geometric and predicate backends. Predicate agreement is an implementation-equivalence check. The two studies test coupled-budget representation and action repair separately; neither isolates a geometric advantage over an equally instrumented predicate.

The matched 512-case joint-risk suite contains boundary-active four-dimensional risk vectors. Every coordinate remains below its independent threshold, but some vectors exceed the shared joint budget. The matched 256-case action suite supplies the same unsafe action and candidate repair to every arm; 192 cases are repairable by deleting an unsup-

ported optional destination and 64 require blocking because the destination is semantically required. No case invokes an external tool or evolves from a completed benchmark trajectory. The matched study is therefore a representation and interface audit rather than a replacement for the end-to-end AgentDojo result. We also audited 614 saved AgentDojo decision records, of which 610 were well formed, by checking the recorded hard margins against an equivalent slack predicate. The two decisions agreed on 610/610 records; 546 were allowed and 64 were blocked or clarified. This is a certificate-consistency audit over saved boundaries, not a new planner replay or an end-to-end utility/ASR result. The matched study is reported separately because system ablations measure the complete defense path, whereas representation controls isolate coupled-risk handling and certificate observability.

Mechanism-aware stress test. To test the execution contract beyond the benchmark’s static injection templates, we ran a frozen 64-case in-process stress suite crossing fresh/stale state with four fixed injection families: forged approval or runtime metadata, policy-override instructions, Chinese webpage instructions, and fabricated prerequisite claims. The attack destination and payload appeared only in

Backend	Safe allow	Unsafe block	False allow
Axis thresholds	487/487	0/25	25/25
Joint geometry	487/487	25/25	0/25
Equivalent predicate	487/487	25/25	0/25

Table 3: Membership under a specified unit-norm risk policy: 487 feasible and 25 infeasible vectors. All arms have zero false blocks. Labels are determined analytically by that policy, rather than independent real-world safety annotations.

Backend + shared repair	Repaired	Required blocks
Geometry	192/192	64/64
Equivalent predicate	192/192	64/64

Table 4: Local action-repair audit: both backends block all 256 original proposals and preserve the intended effect in all 192 repairable cases (192/256 overall). The same transformation and effect check are used in both arms. A decision-only interface has no repair operator and is not treated as an empirical repair competitor.

the untrusted channel, while the trusted state retained only the object handle. The runtime blocked 64/64 attacks, including 32/32 cases with fresh state and 32/32 with stale state; an unprotected executor performed 64/64 injected writes. This is a fixed-family provenance/freshness stress test with a local effect model and no external side effects, not an additional AgentDojo leaderboard result.

Signed-margin behavior. Table 5 summarizes the overall signed margins of 9,083 AgentDojo tool calls across three planner models. Because DAFR admits a call exactly when $M_t(z_t) \geq 0$, the analysis reports the magnitude of the raw signed slack produced by the active constraints. Mean margins are 0.712–0.750 for admitted calls and -0.368 to -0.214 for rejected calls. For each rejected call, the active condition attaining the minimum margin identifies the prerequisite, argument, source, authorization condition, or joint-risk relation used for targeted feedback.

Planner	Calls	$\overline{M}_{\text{adm}}$	$\overline{M}_{\text{rej}}$
DeepSeek V4 Flash	3947	0.712	-0.368
GPT-5.4-mini	2916	0.748	-0.214
Claude Haiku 4.5	2220	0.750	-0.289

Table 5: Mean signed margins for admitted and rejected AgentDojo calls.

Related Work

Prompt-injection attacks and defenses. Indirect prompt injection allows untrusted observations to redirect tool-using agents, as shown by early attacks and later benchmarks (Greshake et al. 2023; Liu et al. 2023; Zhan et al. 2024; Debenedetti et al. 2024). Context-level defenses preserve instruction boundaries, prioritize trusted instructions, or sepa-

rate instructions from data (Hines et al. 2024; Wallace et al. 2024; Chen et al. 2025). PiGuard detects injected content, Task Shield checks task alignment, and MELON compares normal execution with masked re-execution (Li et al. 2025b; Jia et al. 2025; Zhu et al. 2025).

Runtime controls for tool-using agents. AgentSpec and ShieldAgent express runtime requirements through explicit safety policies (Wang, Poskitt, and Sun 2025; Chen, Kang, and Li 2025). Progent enforces privileges over tool names and arguments, and DRIFT checks deviations from a task-grounded function trajectory (Shi et al. 2025; Li et al. 2025a). CaMeL separates control and data flows derived from a trusted request, and Fides enforces confidentiality and integrity labels (Debenedetti et al. 2025; Costa et al. 2025).

Tool-safety evaluation. ToolEmu and ToolSword study failures arising from consequential tool use (Ruan et al. 2024; Ye et al. 2024). AgentHarm and MobileSafetyBench extend evaluation to harmful multi-step behavior and mobile-device control, and MCPTox studies tool poisoning in MCP servers (Andriushchenko et al. 2025; Lee et al. 2026; Wang et al. 2026). Our evaluation uses AgentDojo and ASB to measure task utility and attack-induced tool effects.

Geometric safety formulations. Action shielding restricts decisions to actions that satisfy formal safety specifications (Alshiekh et al. 2018). GIF uses model Jacobians and local output geometry to bound information flow inside language models (Storek et al. 2026). DAFR instead places explicit action–evidence relations at the execution boundary. State-conditioned halfspaces and joint-risk regions combine task support, grounding, provenance, authorization, effects, and state in one feasibility decision, and active margins provide relation-specific revision targets.

Conclusion

DAFR connects tool-call arguments to their supporting evidence and evaluates the resulting action–evidence point against state-dependent execution requirements. Its geometric representation composes joint-risk budgets with grounding and authorization constraints, while a shared certificate records the limiting relations and supports action revision. The retained benchmark results show a useful security–utility trade-off under static attacks. Matched controls establish the difference between independent thresholds and a joint budget, and confirm decision and repair equivalence with a predicate supplied with the same information and repair procedure. The contribution is a structured execution interface whose decisions and revision targets can be inspected together.

Limitations

DAFR depends on correct field bindings, explicit policy templates, and trusted provenance and authorization channels. Schema validity and region membership establish compliance with the encoded policy; neither proves that the policy or the encoder fully captures real-world safety. New effects may require manual bindings or templates. Fixed coefficients

avoid online fitting but do not by themselves demonstrate robustness to parameter choice or eliminate development-set selection effects.

The benchmark evaluation uses fixed static attacks. The 64-case stress suite is a local, fixed-family test and does not establish resistance to adaptive, feedback-driven attackers. The 512-vector study uses analytically specified policy labels, and the 256-action repair suite varies a narrow optional-destination transformation. The 610-record audit checks saved margins rather than newly executed trajectories. These studies establish controlled mechanism behavior, not generalization to arbitrary tools or repair tasks.

The retained multi-model tables have heterogeneous archival provenance, including document-derived baseline aggregates. Utility differences are descriptive and lack paired significance tests. The observed zero attack counts are limited to the evaluated cases; correlated attacks on shared tasks further limit population-level interpretation of binomial intervals. Full end-to-end latency, parameter sensitivity, and an independent adaptive-attack evaluation remain unmeasured in this manuscript.

References

- Alshiekh, M.; Bloem, R.; Ehlers, R.; Könighofer, B.; Niekum, S.; and Topcu, U. 2018. Safe Reinforcement Learning via Shielding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32.
- Andriushchenko, M.; Souly, A.; Dziemian, M.; Duenas, D.; Lin, M.; Wang, J.; Hendrycks, D.; Zou, A.; Kolter, Z.; Fredrikson, M.; Winsor, E.; Wynne, J.; Gal, Y.; and Davies, X. 2025. AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents. In *The Thirteenth International Conference on Learning Representations*.
- Chen, S.; Piet, J.; Sitawarin, C.; and Wagner, D. 2025. StruQ: Defending Against Prompt Injection with Structured Queries. In *34th USENIX Security Symposium*, 2383–2400.
- Chen, Z.; Kang, M.; and Li, B. 2025. ShieldAgent: Shielding Agents via Verifiable Safety Policy Reasoning. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, 8313–8344. PMLR.
- Costa, M.; Köpf, B.; Kolluri, A.; Pavard, A.; Russinovich, M.; Salem, A.; Tople, S.; Wutschitz, L.; and Zanella-Béguelin, S. 2025. Securing AI Agents with Information-Flow Control. arXiv:2505.23643.
- Debenedetti, E.; Shumailov, I.; Fan, T.; Hayes, J.; Carlini, N.; Fabian, D.; Kern, C.; Shi, C.; Terzis, A.; and Tramèr, F. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813.
- Debenedetti, E.; Zhang, J.; Balunovic, M.; Beurer-Kellner, L.; Fischer, M.; and Tramèr, F. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems*, volume 37, 82895–82920.
- Greshake, K.; Abdelnabi, S.; Mishra, S.; Endres, C.; Holz, T.; and Fritz, M. 2023. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. arXiv:2302.12173.
- Hines, K.; Lopez, G.; Hall, M.; Zarfati, F.; Zunger, Y.; and Kiciman, E. 2024. Defending Against Indirect Prompt Injection Attacks With Spotlighting. arXiv:2403.14720.
- Jia, F.; Wu, T.; Qin, X.; and Squicciarini, A. 2025. The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 29680–29697.
- Lee, J.; Hahm, D.; Choi, J. S.; Knox, W. B.; and Lee, K. 2026. MobileSafetyBench: Evaluating Safety of Autonomous Agents in Mobile Device Control. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 37565–37573.
- Li, H.; Liu, X.; Chiu, H.-C.; Li, D.; Zhang, N.; and Xiao, C. 2025a. DRIFT: Dynamic Rule-Based Defense with Injection Isolation for Securing LLM Agents. In *Advances in Neural Information Processing Systems*, volume 38, 83262–83290.
- Li, H.; Liu, X.; Zhang, N.; and Xiao, C. 2025b. PiGuard: Prompt Injection Guardrail via Mitigating Overdefense for Free. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 30420–30437.
- Li, H.; Wen, R.; Shi, S.; Zhang, N.; and Xiao, C. 2026. AgentDyn: A Dynamic Open-Ended Benchmark for Evaluating Prompt Injection Attacks of Real-World Agent Security System. arXiv:2602.03117.
- Liao, Z.; Mo, L.; Xu, C.; Kang, M.; Zhang, J.; Xiao, C.; Tian, Y.; Li, B.; and Sun, H. 2025. EIA: Environmental Injection Attack on Generalist Web Agents for Privacy Leakage. In *The Thirteenth International Conference on Learning Representations*.
- Liu, Y.; Deng, G.; Li, Y.; Wang, K.; Wang, Z.; Wang, X.; Zhang, T.; Liu, Y.; Wang, H.; Zheng, Y.; and Liu, Y. 2023. Prompt Injection Attack against LLM-Integrated Applications. arXiv:2306.05499.
- Ma, X.; Li, T.; Xiao, C.; Yu, Z.; Zhang, N.; and Vorobeychik, Y. 2026. AutoDojo: Adaptive Attacks Expose Superficial Defenses and User-Underspecification Limits in LLM Agents. arXiv:2606.15057.
- Ruan, Y.; Dong, H.; Wang, A.; Pitis, S.; Zhou, Y.; Ba, J.; Dubois, Y.; Maddison, C. J.; and Hashimoto, T. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *The Twelfth International Conference on Learning Representations*.
- Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, volume 36.
- Shi, T.; He, J.; Wang, Z.; Wu, L.; Li, H.; Guo, W.; and Song, D. 2025. Progent: Programmable Privilege Control for LLM Agents. arXiv:2504.11703.
- Storek, A.; Holzer, N.; Zhang, Z.; and Jana, S. 2026. GIF: Locally Sound Geometric Information Flow Control for LLMs. arXiv:2606.23277.

Wallace, E.; Xiao, K.; Leike, R.; Weng, L.; Heidecke, J.; and Beutel, A. 2024. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. arXiv:2404.13208.

Wang, H.; Poskitt, C. M.; and Sun, J. 2025. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. arXiv:2503.18666.

Wang, Z.; Gao, Y.; Wang, Y.; Liu, S.; Sun, H.; Cheng, H.; Shi, G.; Du, H.; and Li, X. 2026. MCPTox: A Benchmark for Tool Poisoning on Real-World MCP Servers. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 35811–35819.

Wang, Z.; Siu, V.; Ye, Z.; Shi, T.; Nie, Y.; Zhao, X.; Wang, C.; Guo, W.; and Song, D. 2025. AGENTVIGIL: Automatic Black-Box Red-teaming for Indirect Prompt Injection against LLM Agents. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 23159–23172.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*.

Ye, J.; Li, S.; Li, G.; Huang, C.; Gao, S.; Wu, Y.; Zhang, Q.; Gui, T.; and Huang, X. 2024. ToolSword: Unveiling Safety Issues of Large Language Models in Tool Learning Across Three Stages. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2181–2211.

Zhan, Q.; Liang, Z.; Ying, Z.; and Kang, D. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, 10471–10506.

Zhang, H.; Huang, J.; Mei, K.; Yao, Y.; Wang, Z.; Zhan, C.; Wang, H.; and Zhang, Y. 2025. Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents. In *The Thirteenth International Conference on Learning Representations*.

Zhu, K.; Yang, X.; Wang, J.; Guo, W.; and Wang, W. Y. 2025. MELON: Provable Defense Against Indirect Prompt Injection Attacks in AI Agents. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, 80310–80329. PMLR.